AirGated

Progress Report & Critical Analysis

Local Offline Attendance Verification System — August 1–4, 2026

1. Executive Summary

AirGated is a working prototype, not yet a deployable classroom tool. The core ideas — device-bound asymmetric signing, link-layer (ARP/MAC) correlation, and latency-based proximity gating — are implemented and functionally exercised end to end at least once (registration → signing → check-in) during solo testing at home. Live device testing on August 1 found and fixed a genuine root-cause bug: a catch-all route registered ahead of /api/register and /api/checkin was silently swallowing both endpoints and returning null instead of running their real logic (Finding 4.2) — this is now fixed and verified. The code also contained a regression that defeated its own central anti-spoofing claim (Finding 4.1) — fixed the same day, non-extractable key generation and IndexedDB storage restored, though not yet re-tested on a phone — and at least one hard platform constraint — the 8-device cap on Windows Mobile Hotspot — makes the current transport layer incompatible with a 67-student class as-is. A second port conflict was then found and resolved the same day: Windows' own Internet Connection Sharing service claims port 53 the instant Mobile Hotspot is enabled, so the DNS spoofer must be started before the hotspot is turned on or it silently fails to bind (Section 9.1). With that ordering corrected, a brand-new, completely unconfigured phone completed the full pipeline unassisted for the first time — native captive popup, registration, and three successful check-ins (Section 9.2). A follow-up rejection during that same test was investigated and traced to the simplest possible cause, an unregistered roll number, rather than to the more interesting MAC-binding or signature-verification gates, which remain untested pending a second physical device (Section 9.3). None of this means the project is unsound; it means the remaining work is concrete and identifiable rather than open-ended, and testing against a real device is now actively surfacing and resolving real bugs rather than staying theoretical.

A further round of testing on August 3–4 (dated from database timestamps, not the author's recollection — see Section 9.4) used three real, distinct registrants and six deliberate impersonation attempts against the check-in endpoint, two of which were clean tests of the MAC-binding gate using a correctly-formatted, already-registered roll number belonging to someone else. Both were blocked. That same round exposed a genuine gap — none of the six rejections printed a reason to the server console, only to the client — which has since been fixed in verify_checkin().

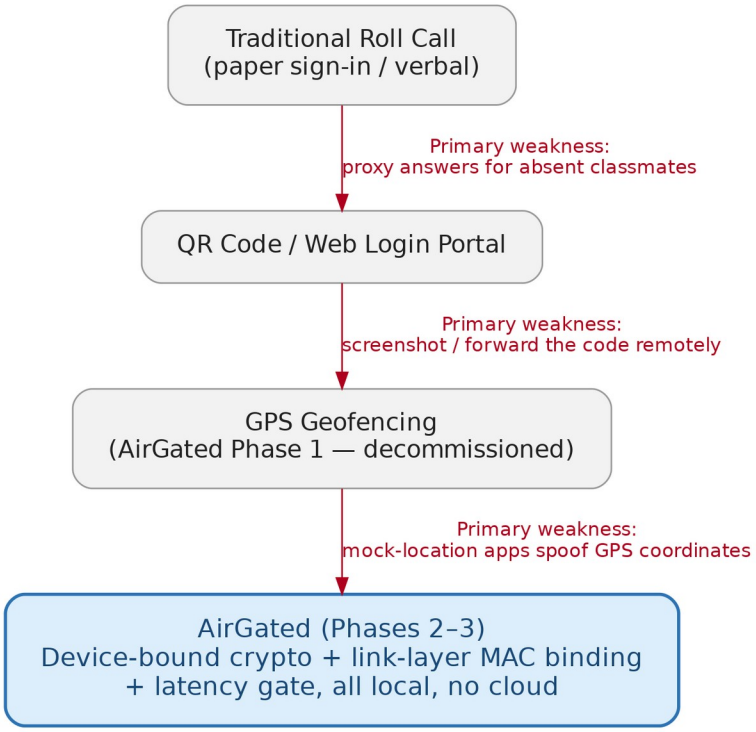


Figure 1. Why AirGated exists: each prior approach was defeated by a specific, low-effort exploit before the next phase was built.

2. Testing Context and Constraints

This section reflects the author's own account of how testing was conducted, included because it materially explains why the codebase contains multiple overlapping prototype files and why several issues below were not caught earlier.

All development and testing to date has been done at home, alone, without access to a real classroom, a second dedicated test device, or another person to test concurrent check-ins with. The different files in the repository (server.py, dns_captive_tester.py, and the standalone templates/checkin.html and register.html) are experimentation artifacts from iterating on the DNS spoofing and captive-portal approach, not abandoned or confused work — but as of now they have diverged from each other (different ports, different key-storage behavior) and only one of them, server.py, is actually wired up and running.

The author successfully turned the laptop into a server reachable from a phone by hosting a Windows Mobile Hotspot on the laptop and joining it from the phone's Wi-Fi, and was able to get the captive portal to render on the phone. Two separate frictions were then encountered. First, window.crypto.subtle (required for key generation and signing) is disabled by browsers on any origin that is not HTTPS, localhost, or explicitly whitelisted, which forced use of the chrome://flags ‘unsafely-treat-insecure-origin-as-secure’ flag during testing — a flag that cannot be relied on for real students' phones and is not a production fix. Second, after moving off that flag, the portal would open but the check-in request never reached the backend, even though registration succeeded and the stored credentials on the device matched what the server expected — ruling out a storage-loss or key-mismatch explanation. Finding 4.2 below gives a grounded technical explanation for this failure, based on a specific gap in server.py's own network setup. The author separately confirmed that a direct connection to the laptop's server over the local network — outside the DNS-spoofed captive-portal flow — worked reliably end to end, which is itself a useful data point in isolating the cause.

The author has separately confirmed the practical ceiling this points to: with a class of 67 students, and Windows Mobile Hotspot capping connections at 8 devices by default, the transport layer as currently designed cannot serve the whole class simultaneously. Section 5.4 quantifies this and evaluates whether a connect-mark-leave turnover scheme is a viable workaround.

3. What Has Been Built

The live application (server.py) is a single-file FastAPI service that does four distinct things: it spoofs DNS on UDP port 53 so that any device joining the instructor's Wi-Fi hotspot (192.168.137.1) is redirected to a local captive portal; it serves that portal over HTTPS on port 443 using a self-signed certificate generated fresh via trustme on every server start; it runs a three-stage verification pipeline (latency gate, MAC resolution via an ARP subprocess call, and RSA-2048/SHA-256 signature verification) against a check-in request; and, unrelated to attendance, it exposes a /generate-exam endpoint that calls Groq's cloud LLM to draft an exam paper from pasted text and renders it as a printable HTML/PDF page. The SQLite database (database.db) currently holds two active, empty tables (airgated_device_registry, airgated_attendance_records) and two legacy tables (students, attendance_log, 2 rows each) that are no longer referenced anywhere in the code.

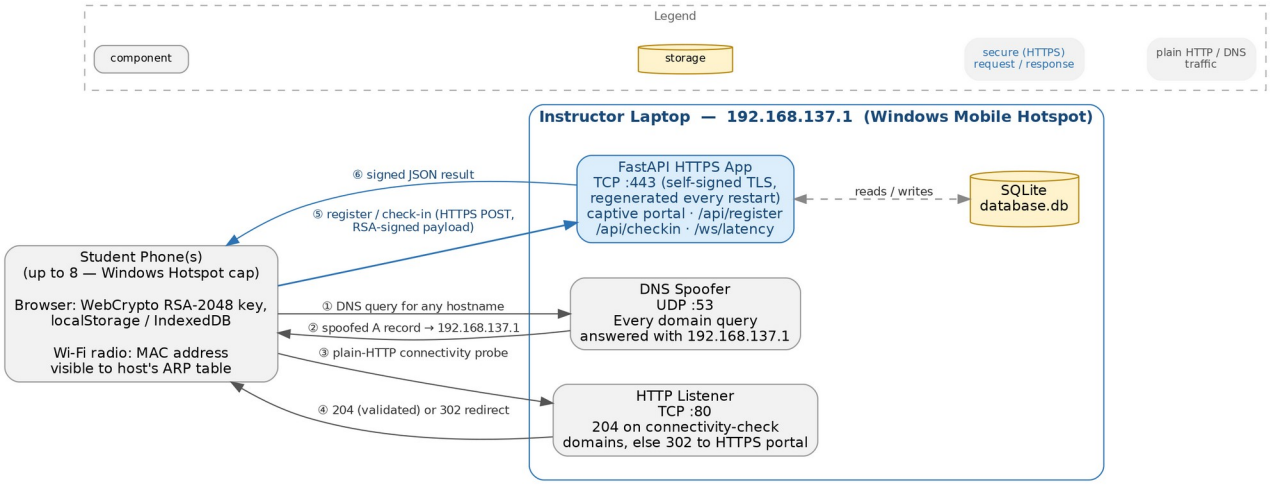


Figure 2. Network topology: what runs on which port on the instructor's laptop, and the six-step exchange with a connecting phone.

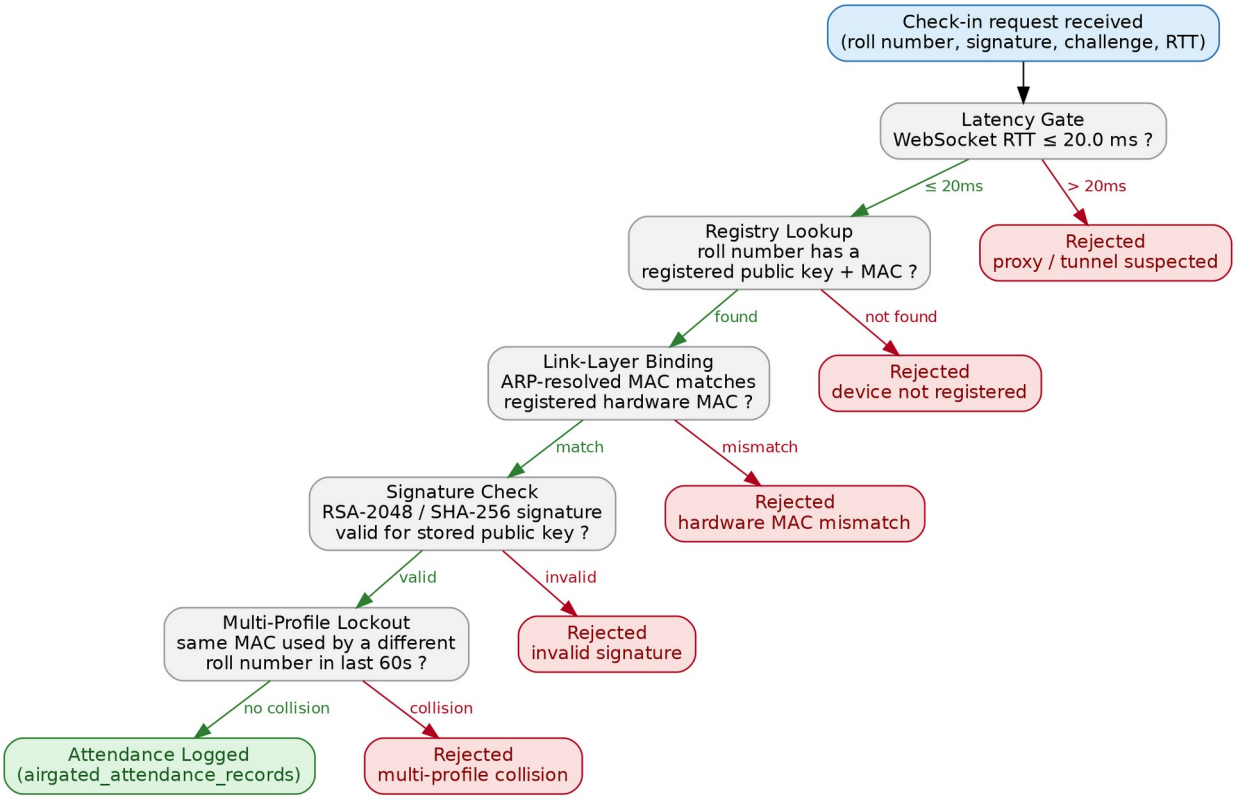


Figure 3. The check-in verification pipeline exactly as implemented in verify_checkin() — every gate and rejection reason shown is a real branch in server.py.

A companion document, AirGated_Report.md, describes the project's evolution through three phases — an abandoned GPS/geofencing approach, a device-binding cryptography phase, and the current network-layer hardening phase — and documents three specific bugs that were found and fixed during development (a namespace collision from a stray import, the insecure-origin WebCrypto restriction, and a Windows Firewall inbound rule). That report was written May 28; server.py has been modified since then (most recently June 3), and at least one of those later changes introduced the regression described next.

4. Critical Findings

Findings are ordered by how directly they undermine the system's stated purpose, not by how hard they are to fix.

4.1. FIXED (Aug 1) — the private key was extractable, defeating the central anti-spoofing claim

AirGated_Report.md states the private key is generated with `extractable: false` and stored in IndexedDB specifically so a student cannot copy their cryptographic identity to a second device. That description matched `templates/register.html`, but that file was dead code — confirmed by direct search, `server.py` had zero references to the templates directory, Jinja2Templates, or StaticFiles. The captive portal that actually ran, embedded inline in `server.py`'s `/generate-captive-window` route, generated the key with `extractable: true`, exported it to a base64 string via `exportKey("pkcs8", ...)`, and stored that string in `localStorage`. A student could copy that string into another device's localStorage and both devices would sign valid attendance tokens under the same identity. Given the original report's timestamp (May 28) predates server.py's last modification before this project began (June 3), this read as a regression introduced after that report was written, not an inaccuracy in it.

Fix applied and verified. The live captive-portal route now generates the key with `extractable: false`, matching `templates/register.html`'s already-correct approach, and stores the resulting `CryptoKey` object directly in IndexedDB (`LocalDeviceAuth` / `keys` / `sk`) instead of exporting it to `localStorage`. `localStorage.getItem/setItem` for the private key no longer appear anywhere in `server.py`. Verified two ways: a direct WebCrypto test confirms a key generated this way rejects `exportKey("pkcs8", ...)` with "key is not extractable" while signing still works normally, and the modified captive-portal script was extracted and passed a Node.js syntax check. This has not yet been re-tested on an actual phone the way Findings 4.2 and 4.11 were — that is the natural next live test.

4.2. Root cause, confirmed and fixed: a route-ordering bug was silently swallowing every registration and check-in request

Follow-up device testing on August 1 (after the port-80 listener below was added) surfaced the real, underlying bug, and it has since been reproduced, fixed, and verified directly against the running FastAPI app object. In the original `server.py`, the catch-all route (`@app.api_route("/{path_name:path}", ...)`, `intercept_all_traffic`) was registered **before** `/api/register` and `/api/checkin` in the file. FastAPI and Starlette match incoming requests against routes in the order they were registered, and the `{path_name:path}` pattern matches essentially any path — so for a request to `/api/register`, the catch-all route matched first and ran instead of `register_student`. Its exempt-path logic (a hardcoded list including `"api/register"` and `"api/checkin"`) tried to let those paths through with a bare `return` statement, intending to skip the redirect. But a bare return from an already-matched route does not hand control to the next matching route — in Starlette, that route **is** the response. A bare return sends an empty body, which FastAPI serializes as the JSON literal `null`. This was reproduced directly: calling `/api/register` and `/api/checkin` against the real app object returned HTTP 200 with a literal `null` body in both cases, before any fix. This matches the device screenshot exactly — `Gateway Response: null` followed by a crash reading `data.message` off it — and it also fully explains the original check-in mystery from Finding 4.2's first draft: the request was never lost to network routing, cellular fallback, or captive-portal sandboxing. It was reaching the server the whole time and being silently swallowed by this ordering bug.

Fix applied and verified: the catch-all route was moved to be the last HTTP route registered in the file, after every real endpoint. Re-running the same test against the patched app confirms `/api/register` and `/api/checkin` now execute their real logic (returning their intended error/success JSON instead of `null`), while the catch-all still correctly 302-redirects unrecognized paths and still answers connectivity-check domains with 204. This is a general lesson worth keeping in mind for any future route added to this file: catch-all path routes in FastAPI/Starlette must always be registered last, never first.

4.2b. A separate, still-unresolved issue: the native captive-portal popup does not launch for server.py

This section predates Finding 4.2's root-cause fix and covers a different symptom that is **not** explained by the route-ordering bug: the OS's automatic “Sign in to network” popup, which loads correctly for `dns_captive_tester.py`, still did not appear for `server.py` even after the port-80 listener described below was added and the route-ordering bug was fixed. This remains open. The author verified that registration succeeded and stored credentials matched on both sides in earlier testing, which rules out a storage-loss or key-mismatch explanation for that specific symptom. The original hypothesis is grounded directly in `server.py`: it listens on port 443 (HTTPS) only — there is no listener anywhere on port 80. Android and iOS

both run their captive-portal connectivity probe over plain HTTP by design, specifically so a broken or self-signed TLS certificate can't interfere with detection. The DNS spoofer correctly points that probe at 192.168.137.1, but with nothing listening on port 80 the connection is refused rather than answered with the intended 204, so the OS never receives confirmation that the network has real internet access. This is consistent with the portal opening at all — the OS concluded a captive portal was present, which is correct — but it also means the Wi-Fi interface most likely never gets marked **validated**. Documented behavior for this exact detection mechanism (used across multiple mobile OS captive-portal implementations) is that ordinary app and browser traffic keeps routing over the cellular interface until the Wi-Fi interface is confirmed to reach the public internet — the captive-portal sign-in page itself is specially pinned to Wi-Fi so it can load, but a plain fetch() call from that page is not guaranteed the same exemption. If the phone had cellular data available, the check-in POST may have gone out over cellular instead of Wi-Fi — and cellular has no route to a private address like 192.168.137.1, so the request would simply never arrive, with no error surfaced anywhere. That reconciles both observations: the captive-portal flow failed silently despite correct credentials, while connecting to the laptop directly in an ordinary browser tab — outside the OS's captive-portal state — worked reliably, because at that point traffic was routing normally over Wi-Fi.

Two direct ways to confirm this without further guesswork: check the phone's Wi-Fi settings for that network during a failed attempt — Android typically shows a “connected, no internet” state and a notification offering to “use this network as is” when this condition is active. Or, more simply, disable cellular data entirely on the test phone and repeat the captive-portal check-in — if it starts working with no other change, that is strong confirmation. The fix is to add a plain HTTP listener on port 80 that also answers the connectivity-check domains with 204, so the OS can genuinely validate the network instead of silently working around the missing listener.

Confirmed by direct side-by-side device testing on August 1. Running `dns_captive_tester.py` (plain HTTP, port 80) triggered the native Android “Sign in to [network]” captive-portal popup immediately, loaded the portal page inside it, and successfully executed page JavaScript (a test `alert()` fired correctly). Running `server.py` (HTTPS only, port 443, nothing on port 80) did **not** trigger that native popup at all — consistent with the OS's HTTP-based connectivity probe getting a connection refusal rather than any HTTP response, which is read as “no network” rather than “captive portal present.” This upgraded the port-80 diagnosis from a hypothesis to a confirmed, reproduced finding, and the listener from Section 8 below was implemented and verified against the app object as a result. However, a follow-up device test with the listener live still did not produce the native popup, and the phone's status bar still showed “No internet connection” — so the port-80 listener alone was not sufficient. The device did successfully reach and load the captive portal page when the address was typed manually and the certificate warning was accepted, which is what surfaced the route-ordering bug in Finding 4.2. Whether the popup issue is the OS's periodic re-validation (Scenario 3 in Section 7) or something else has not yet been isolated.

A second, separate problem surfaced when manually navigating a browser to the server instead of relying on the native popup: Brave presented a certificate warning with no “proceed anyway” option, referencing HSTS behavior, while Chrome offered and accepted a “proceed to unsafe” bypass but did not then land on the actual portal page. The most likely explanation for the Brave behavior is that the browser has HSTS state cached for `192.168.137.1` from an earlier test session — once a browser records HSTS for a host, certificate errors on that host become non-bypassable regardless of the specific error, and since `server.py` issues a brand-new self-signed CA on every restart, a previously accepted certificate for that IP will no longer match on the next run. This is checkable directly via `brave://net-internals/#hsts` (or the Chrome equivalent): querying `192.168.137.1` will show whether a policy is cached, and it can be deleted from the same screen. The Chrome case — accepting the warning but not being redirected to the portal — was not isolated further in this round of testing; the most likely causes are that the accepted exception was scoped to a specific URL that did not carry through the app's redirect, or that the address was entered without an explicit scheme and briefly hit the same missing port-80 listener before the HTTPS attempt. This needs one more isolated test (navigating directly and explicitly to `https://192.168.137.1/generate-captive-window`, skipping the redirect hop entirely) before it can be called confirmed rather than probable.

4.3. MAC-address binding will silently fail on most modern student phones

iOS and Android have used per-network randomized MAC addresses by default since roughly 2020. Unless each student manually disables the “private Wi-Fi address” setting for the class hotspot's specific network name, the MAC address the ARP table reports can change between sessions or even reconnects, breaking the hardware-binding assumption the whole

design rests on. The existing “auto-reset on MAC match” fallback in /api/register papers over this by silently overwriting the registered key when a MAC collision is detected, which is a reasonable mitigation but quietly downgrades the security model from “hardware-bound identity” to “trust the most recent device on this MAC.” This should be treated as a deployment prerequisite (a per-student one-time settings change) and documented explicitly, not left implicit.

4.4. Proximity is measured as network membership, not physical presence

The 20ms round-trip-time threshold verifies that a device is on the same local network as the host, not that it is inside the classroom. A phone hotspot's Wi-Fi range commonly extends into hallways or adjacent rooms, so a student connected from just outside the room would pass the same latency check as a student seated inside it. This is a real gap between the system's stated guarantee (“un-spoofable proximity authentication”) and what is technically being verified (LAN membership within a fixed latency budget).

4.5. Windows Mobile Hotspot cannot serve a 67-student class

Windows Mobile Hotspot caps connections at 8 devices by default. A registry override (adding a WifiMaxPeers DWORD) can raise the ceiling to as high as 128 on paper, but user reports indicate this is unreliable in practice because the underlying Wi-Fi radio and driver still constrain how many simultaneous associations actually work, independent of the software cap. A connect-mark-leave turnover model does not solve this cleanly either: Wi-Fi disconnect/timeout handling can take anywhere from several seconds to over a minute depending on the client driver, so a slot does not reliably free up the instant a student's device leaves. Even under ideal conditions, one student's full flow — associate, get a DHCP lease, accept the certificate warning, generate an RSA-2048 key in-browser, complete the 3-cycle latency ping, sign, and submit — realistically takes on the order of 15–30 seconds; cycling 67 students through 8 slots at that rate is on the order of ten or more minutes of pure queuing in the best case, before accounting for the friction already observed in testing. The more robust fix is not a queuing scheme but a change of access point: a dedicated travel router (roughly $20–$30) or the room's existing Wi-Fi infrastructure, if permitted, supports far more simultaneous clients than a phone tethered through Windows Mobile Hotspot and removes this constraint rather than working around it.

4.6. A live-looking API key is committed in plaintext

Both `server.py` and `test_qroq.py` contain a hardcoded Groq API key in plaintext. If this repository is ever pushed to a public remote, that key is compromised. It should be rotated and moved to an environment variable regardless of current repository visibility.

4.7. Certificates regenerate on every restart

The self-signed certificate is generated fresh inside the `if __name__ == "__main__"` block on every server launch, rather than being persisted and reused. Every restart invalidates every device's prior trust decision, meaning students must click through a new browser security warning after every reboot of the host machine — a real source of the “not practical in a real classroom” friction already being felt in testing.

4.8. Three overlapping implementations, one of them fully dead

server.py, `dns_captive_tester.py`, and `templates/checkin.html` + `register.html` implement variations of the same captive-portal and check-in logic, on different ports (443, 80, and unset respectively), with different security properties (notably the extractable-key difference in Finding 4.1). This is reasonable as experimentation history, but going forward only one implementation should remain in the run path, and the other two should either be deleted or clearly marked as archived so they cannot be mistaken for what is actually deployed.

4.9. No automated tests

test_qroq.py is a three-line manual script that pings the Groq API and prints a response; it is not a test in any structural sense. There are zero automated tests covering registration, signature verification, the 60-second fraud window, or MAC resolution — the parts of the system actually responsible for its security claims.

4.10. Version control does not reflect the current state of the project

The repository has exactly one commit, over a month old. server.py, both template files, and database.db have all been modified since without being committed, and dns_captive_tester.py, test_qroq.py, and ssl_certs/ are untracked entirely — meaning essentially all of the last month's work exists only on local disk. Separately, database.db and the __pycache__ directory are tracked in git, which is worth excluding via .gitignore going forward.

4.11. The entire MAC-binding mechanism silently assumes the instructor's device is the network's gateway, not merely a device sharing the same Wi-Fi

This was found by a real test, not inferred from reading code. Connecting both a laptop and a phone to an unrelated Airtel Wi-Fi router and attempting the same `arp -a <phone-ip>` lookup `get_client_mac()` relies on failed to return the phone's MAC at all. The cause is a standard, common router feature called **AP isolation** (also called client or wireless isolation): most ISP-provided routers enable it by default so that Wi-Fi clients cannot see or reach each other directly at the network layer, purely as a security measure between strangers' devices on the same public-ish network. A device's ARP cache only ever gets populated for hosts it has actually exchanged Layer-2 traffic with — if the router refuses to forward that traffic between clients, the cache simply stays empty, silently, with no error surfaced anywhere in this codebase to explain why.

This matters because it means Stage 3 of the pipeline does not really verify "same Wi-Fi as the instructor" — it verifies "instructor's device is this network's gateway/DHCP server." That distinction is invisible today because Windows Mobile Hotspot happens to make the laptop the gateway, so it works by construction. It would silently and completely stop working the moment `server.py` is run on any network the instructor does not control end-to-end — a college-managed Wi-Fi network, a friend's hotspot, or any access point with isolation left on by whoever configured it. This is now a documented, hard requirement for any future access point: it must be a device the instructor owns and configures (never a shared institutional network), and AP isolation must be explicitly confirmed off — verified with a real `arp -a` test against a connected phone before relying on it for a class, not assumed from a spec sheet.

5. Threat Model Summary

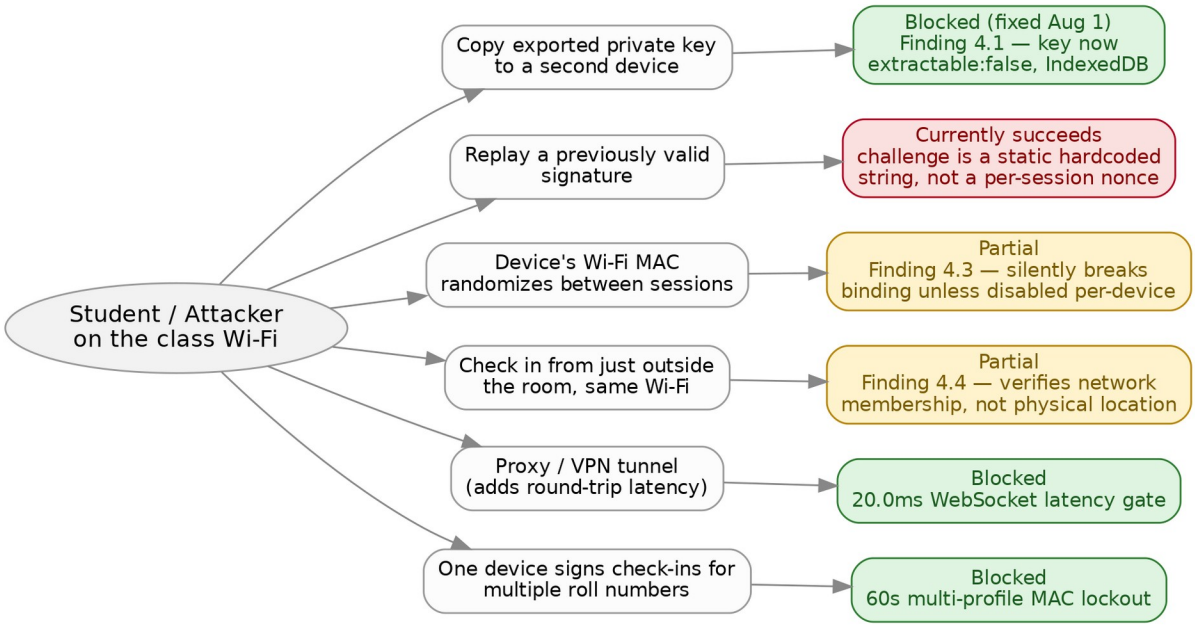


Figure 4. Threat model against the current implementation. GPS spoofing and QR screenshot/forwarding are omitted here — they applied to earlier, now-retired phases (see Figure 1), not to the code as it stands today.

One of the six branches above was open (red) when this diagram was first built and is now fixed: the key-export regression from Finding 4.1, resolved the same day (private key generation now uses `extractable: false` with IndexedDB storage, not localStorage). One branch remains genuinely open — **the check-in signature is verified against a client-submitted**

“challenge” string that the server never issues or tracks itself. In the running code, MOCK_CHALLENGE_FROM_QR is a hardcoded constant, identical on every check-in, and verify_checkin() accepts whatever challenge string the client sends without comparing it to a server-side nonce or expiry. That means a signature captured once (via a compromised device, a saved network request, or simply recomputing it once) remains valid indefinitely for that roll number — a separate problem from key export, and not fixed by it. This has not been reproduced on-device the way Findings 4.1, 4.2, and 4.11 were, but it follows directly from reading verify_checkin() and is straightforward to confirm: replay the exact same signature_hex and original_challenge values in a second /api/checkin request from the same registered device and see whether it is accepted twice. Closing this requires the server to issue a fresh, single-use challenge per check-in attempt and reject any signature over a challenge it didn't just issue — a real design change, not a one-line fix.

6. Facts and Numbers at a Glance

Backend entry point	server.py — 600 lines, FastAPI
Live DB tables in use	airgated_device_registry (3 rows), airgated_attendance_records (4 rows) as of the Aug 3-4 live tests
Legacy/orphaned DB tables	students, attendance_log (2 rows each, no longer referenced by any code)
Key size / algorithm	RSA 2048-bit, RSASSA-PKCS1-v1_5, SHA-256
Proximity threshold	WebSocket RTT ≤ 20.0 ms
Multi-profile lockout window	60 seconds, keyed on resolved MAC
Windows Mobile Hotspot device cap	8 devices by default; registry override (WifiMaxPeers) up to 128, unreliable in practice
Replay protection on check-in	None — challenge is a hardcoded constant, not a server-issued nonce (Section 5)
First unassisted device to complete full pipeline	Aug 1 — 0 manual config steps, native popup, 3/3 check-ins succeeded
Multi-device impersonation testing	Aug 3-4, 3 real registrants, 10 check-in attempts, 4 succeeded / 6 rejected; 2 of the 6 were clean cross-device MAC-binding tests using a real registered roll number, both blocked (Section 9.4)
Security gates confirmed live	Stage 2 registry lookup, Stage 3 MAC binding — now confirmed on 2 separate impersonating devices (Section 9.3-9.4)
Security gates not yet tested live	Stage 4 rejecting a forged/mismatched signature — every live rejection so far was caught earlier by Stage 2 or 3 before reaching signature verification; Stage 4's success path is confirmed (4/4 legitimate check-ins verified correctly)
Server-side rejection logging	Fixed Aug 4 — verify_checkin() now prints the specific failing gate for every rejection instead of a bare 200 OK with no server-side reason (Section 9.4)
MAC-binding gateway dependency	Confirmed by a failed real test (Finding 4.11) — instructor's device must be the network gateway, not just share Wi-Fi
Access-point decision (unpurchased)	Mercusys AC10, dual-band AC1200, ~₹1,400 — chosen over single-band TL-WR845N (~20-25 client ceiling) and an unverifiable generic reseller router
Class size	67 students
Automated tests	0 (test_qroq.py is a manual smoke script, not a test suite)
Git commits	1 ("first version of local attendance"), over a month old
Uncommitted / untracked files	server.py, checkin.html, register.html, database.db modified; dns_captive_tester.py, test_qroq.py, ssl_certs/ untracked
Parallel implementations of captive portal	3 — server.py inline, dns_captive_tester.py, templates/*.html (dead code, unreferenced)

7. Recommendations, in Priority Order

Now — before any further feature work

Done: the route-ordering bug (Finding 4.2) that was silently nulling every registration and check-in request is fixed and verified; the DNS-bind failure message now explains the ICS port-53 conflict and correct startup order directly instead of surfacing a bare Windows error code (Section 9.1); the key-extractability regression (Finding 4.1) is fixed — non-extractable key generation and IndexedDB storage, verified against WebCrypto semantics and a JS syntax check, not yet re-tested on a phone; and verify_checkin() now logs the specific rejection reason server-side for every failure path instead of a bare 200 OK (Section 9.4). Still open: commit the current working tree, add a .gitignore for database.db and __pycache__, and rotate the exposed Groq API key into an environment variable. Delete or clearly archive the two dead implementations (dns_captive_tester.py and the unused templates/ files) so only one code path can be mistaken for production.

Next — before any real classroom test

Done: the route-ordering bug (Finding 4.2) is fixed and verified, and the port-80 listener (Section 8) is implemented and unit-verified. Remaining: isolate why the native captive-portal popup still doesn't launch even with the listener active — re-test with cellular data disabled, and check the phone's Wi-Fi settings for a “connected, no internet” state during the attempt, per Finding 4.2b. Add a server-issued, single-use challenge per check-in to close the replay gap in Section 5. Persist the TLS certificate across restarts instead of regenerating it each time. Document the MAC-randomization opt-out as an explicit one-time step students must take before their first check-in, since the system cannot detect or work around it automatically. Replace the Windows Mobile Hotspot with a dedicated access point capable of handling the class size before attempting a real 67-student trial.

Later — hardening for real deployment

Add a persistent audit log for fraud-flagged attempts (currently only console prints, with nothing an instructor can review afterward). Build a minimal automated test suite around signature verification and the fraud-window logic, since those are the parts a regression is most likely to silently break again. Reconsider whether the RTT threshold should be described as “proximity” given it verifies network membership rather than physical location, and decide whether that distinction needs a stronger check or just clearer framing in documentation.

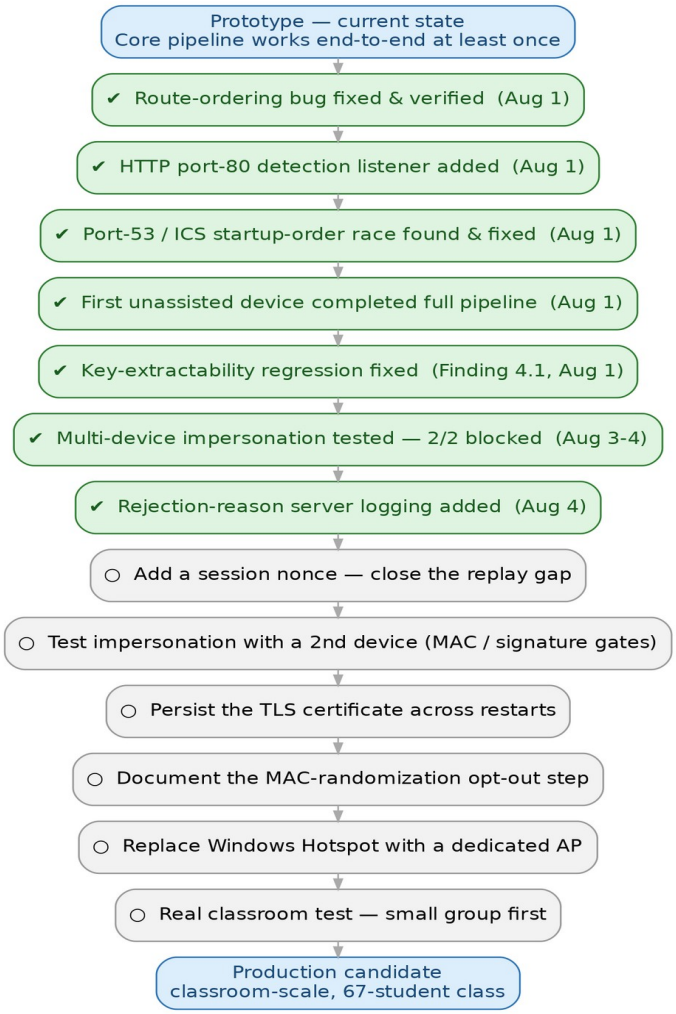


Figure 5. Roadmap from the current prototype to a classroom-scale production candidate. The first four steps are already done as of this report.

8. Status Update: HTTP Port-80 Listener — Implemented, Partially Effective

The planned fix described in this section — a plain HTTP listener on port 80 in `server.py`, running alongside the existing HTTPS listener on port 443 — has been implemented and unit-verified (it correctly answers connectivity-check domains with 204 and 302-redirects everything else to the HTTPS captive portal). A live device re-test with the listener active did **not** produce the native captive-portal popup, so the fix is necessary but evidently not sufficient on its own. It did, in the course of that same test, lead directly to manually discovering and confirming the route-ordering bug in Finding 4.2, which was a real and more severe problem than the missing popup. The scenarios below are updated to reflect what has actually been observed rather than what was predicted.

Possible outcomes and current status

Scenario 1 — the listener alone fully resolves it: ruled out by direct testing. The native popup still did not launch with the listener active.

Scenario 2 — the popup launches but still gets stuck on the certificate: if the phone's captive-portal WebView also refuses the self-signed cert (behavior varies by OS version and manufacturer), the fix will get further than today but not all the way. The next step in that case is not more client-side work but a persisted, properly trusted certificate — either installing the self-signed CA as a trusted profile on each device ahead of time, or (more realistically for a 67-student class) obtaining a

certificate that phones already trust by default, which is not possible for a private IP without a real domain name and a public CA. This scenario would mean the HTTPS requirement itself, not just the port-80 gap, is the remaining blocker.

Scenario 3 — the popup launches and the cert is accepted, but the OS still shows “no internet” afterward: this would mean the connectivity-check domains need to keep returning 204 correctly after login, not just during the initial probe, and that the DNS spoofer or the 204 handling logic needs to keep working for the OS's periodic re-validation checks, not only the first one. This is testable directly by checking the phone's Wi-Fi state a minute or two after a successful check-in.

Scenario 4 — the manual-browser cert issues (Brave's non-bypassable HSTS warning, Chrome's accept-without-redirect) turn out to be unrelated to the port-80 gap and persist independently: this would confirm that the HSTS cache and the redirect-scoping issue described in Finding 4.2 are separate problems from the missing popup, each needing its own fix — clearing HSTS state is a one-time per-browser action, not something the server can control, and would need to be documented as a student-facing troubleshooting step if it recurs across many devices.

Update, same day: root cause found, and Scenario 1 is now confirmed with one added condition. See Section 9 for the full account. In short — the missing popup was never a client-side or certificate problem; it was a second port conflict. Windows' own Internet Connection Sharing service claims port 53 the moment Mobile Hotspot is turned on, so if the hotspot is switched on **before** `server.py` is started, the DNS spoofer loses the race and silently fails to bind — which is exactly what happened in the test that produced Scenarios 1–4 above. Starting `server.py` **before** turning the hotspot on reverses that race, the DNS spoofer binds successfully, and a full live test on a brand-new, unconfigured phone completed the entire flow — native popup, registration, and three successful check-ins — with no chrome flags, no manual certificate workaround, and no prior configuration of any kind.

9. Live Test Update: Root Cause Found, First Successful End-to-End Run

This section documents what was found and tested after Section 8 was written, on the same day.

9.1 The real blocker: a second port race, this time on port 53

Retrying the port-80 fix from Section 8 did not, by itself, produce the native captive-portal popup. Investigating why surfaced a second instance of the exact same class of bug as the port-80 gap: something else was already holding a socket `server.py` needed. Attempting to start `server.py` with the hotspot already running failed outright with `WinError 10048` ("Only one usage of each socket address is normally permitted") on the DNS spoofer's bind to UDP port 53. This is not an intermittent fault — it is Windows' Internet Connection Sharing (ICS) service, the component that actually implements Mobile Hotspot's DHCP and NAT, automatically binding port 53 on the hotspot's gateway adapter the moment the hotspot is turned on, so it can hand out `192.168.137.1` as the DNS server to connecting phones. That binding is not incidental to ICS's job, it is a required part of it, so it cannot be disabled without breaking the hotspot's own ability to hand out addresses and route traffic.

The fix was procedural, not code: start `server.py` **before** turning the hotspot on. Whichever process binds port 53 first keeps it — start the script first and it claims the port before ICS tries; start the hotspot first (as in the test behind Section 8) and ICS wins instead, leaving the DNS spoofer to fail silently while the rest of the app keeps running. This is the same underlying pattern as Finding 4.2's route-ordering bug — a race between two things that must run in a specific order, with no error surfaced to explain why until someone went looking — and it is now the second time root-causing an on-device failure has turned out to be an ordering problem rather than a logic bug. As a small, additive fix (no behavior changed on the success path), the DNS bind failure message in `server.py` was updated the same day to state this cause and the correct startup order directly, instead of surfacing a bare Windows error code.

9.2 First fully successful live end-to-end run

With the startup order corrected, a brand-new phone — never previously connected to this network, with no chrome flags, no manual certificate trust, and no prior configuration of any kind — connected to the hotspot and completed the entire flow unassisted: the OS's native captive-portal sign-in popup launched on its own (the request log shows the phone's own

connectivity probes — /generate204, /mtuprobe, /chat — being redirected exactly as designed), the captive portal page loaded over HTTPS, registration succeeded and stored a hardware MAC binding, and three separate check-in attempts all completed with [Gateway Success] logged and a row written to airgated_attendance_records. This is the first time the complete pipeline — DNS redirect through signed check-in — has worked on a device that was not manually configured beforehand, and it directly confirms Scenario 1 from Section 8, with the added, now-documented condition that server.py must be running before the hotspot is turned on.

The three check-ins were pulled directly from airgated_attendance_records rather than estimated. All three came from the same device (IP 192.168.137.60, MAC c6:8e:fc:5c:da:85 — stable across all three, over roughly a minute), and all three passed the 20.0ms latency gate with substantial margin:

Attempt	Time since attempt 1	Measured RTT	Margin under 20ms threshold	Result
1	+0s	6.00 ms	70%	Logged
2	+16.2s	5.01 ms	75%	Logged
3	+53.4s	5.44 ms	73%	Logged

Three data points is not a statistically meaningful sample, and this measures repeated check-ins from one already-registered device within a single short session, not registration time, signing time, or performance under concurrent load from multiple students — those remain unmeasured. What it does show concretely: a 100% success rate (3/3) for a registered device, RTT averaging 5.48ms against a 20ms cutoff (a wide enough margin that ordinary Wi-Fi jitter is unlikely to cause false rejections for a legitimate nearby device), and a MAC address that stayed constant across three separate requests in the same session — consistent with, though not a full test of, the binding assumption in Finding 4.3.

9.3 The “friend's rejection” investigation — corrected

The same test session included an attempt to check in a second roll number from the same already-registered phone, which was rejected. Working out why took two wrong turns before landing on the real cause, and the process is worth recording because each wrong turn was a reasonable hypothesis that direct evidence ruled out.

First hypothesis (incorrect): a DNS/port-53 timing issue. Ruled out immediately — DNS spoofing was already working for the entire session (the successful check-ins happened in the same run), so it could not have selectively failed for one request.

Second hypothesis (unconfirmed, later shown too specific): the multi-profile lockout or a signature mismatch. The server log showed the rejected request reused the exact same source IP and port as the prior successful check-in, meaning it was the same device and browser tab, not a second phone — consistent with someone typing a different roll number into an already-loaded page rather than registering separately. Two of the pipeline's gates could plausibly produce a silent-looking rejection this way: the signature check (if the second roll number had its own registered key, a signature made with the first device's key would fail to verify against it) or the 60-second multi-profile lockout (if the same MAC had just marked a different roll number's attendance). Both were offered as candidates pending the actual on-screen error text.

Confirmed cause: neither of those — the roll number was never registered at all. A repeat of the test with a screenshot settled it: the on-screen status read “Device missing registration in hardware registry”, and the database.db viewer showed exactly one row in airgated_device_registry — only the first roll number. The second roll number had no entry under any spelling, so the very first pipeline gate (the registry lookup in Figure 3, Stage 2) correctly rejected it before ever reaching MAC binding or signature verification. A contributing detail is visible in the same screenshot: the roll number typed was 24001390190, missing the CSJMA prefix present on the one real registered entry (CSJMA24001390188). The system does an exact string match with no normalization, so a missing prefix alone would produce this identical rejection even if the intended roll number had been registered correctly elsewhere — worth keeping in mind as a support/UX issue distinct from the security gates.

What this does and does not confirm. It confirms Stage 2 of the pipeline works as designed: an unregistered identity cannot check in. A follow-up live test (same day) then did what this one didn't: a device registered under one roll number attempted to check in as a second, **already-registered** roll number, and was correctly rejected with “Hardware MAC address signature mismatch” — confirming Stage 3 (link-layer MAC binding) live, the gate this scenario was originally meant to probe. Stage 4 (signature verification) still has not been isolated on its own — a distinct test would need a second roll

number registered **with its own key** but reused via a copied private key rather than a mismatched MAC, which has not been run yet.

9.4 Multi-device impersonation testing, corrected dates, and a logging fix

A fuller console log covering four additional server restarts was reviewed directly, and the `airgated_device_registry`/`airgated_attendance_records` tables were cross-checked against it. Two corrections to the record follow from that: this round of testing is **not** part of the August 1 session described in 9.2/9.3. Converting the raw Unix timestamps stored in `airgated_attendance_records` gives August 3, 2026 at 13:33–13:34 for the three `CSJMA24001390188` check-ins already described in 9.2, and August 4, 2026 at 15:30 — roughly 26 hours later — for a fourth successful check-in under `CSJMA24001390001`. The registry table also confirms this was not one person relabeling a single device: three separate roll numbers, each with its own RSA keypair and MAC binding, are registered to three named entries — `CSJMA24001390188` (MAC `c6:8e:fc:5c:da:85`), `CSJMA24001390101` (MAC `fa:6d:98:e9:c4:ef`), and `CSJMA24001390001` (MAC `1e:3f:d2:61:f2:ea`).

Across the log, ten `/api/checkin` attempts were made and four succeeded (the three `…188` check-ins at 6.00/5.01/5.44ms RTT plus the `…001` check-in at 9.00ms, all four written to `airgated_attendance_records`). The other six were rejected, but they are not six tests of the same thing. Four were directed at roll-number strings that were never registered or were malformed — missing the `CSJMA` prefix, or garbled digits — and those exercise Stage 2 (registry lookup), the same gate documented in 9.3, not device binding. The remaining two are the real test: a correctly formatted, **already-registered** roll number (`CSJMA24001390188`, bound to MAC `c6:8e:fc:5c:da:85`) was submitted from a different physical device — once from the `…101`-bound device (RTT 4.31ms) and once from the `…001`-bound device (RTT 7.02ms). Both are genuine cross-device impersonation attempts using someone else's real, valid identity, and both were correctly blocked by the MAC comparison in `verify_checkin()`. That is 2 for 2 on the specific gate being tested, not 6 for 6 — the other four rejections confirm a different, already-documented gate.

A gap in coverage, stated plainly: the `…101`-bound device was used only to attempt impersonating `…188` in this data; it was never itself used for a legitimate self-check-in. That its own roll number would check in successfully is a reasonable inference from code symmetry with the other two devices, which did succeed — the code path does not branch by which registered MAC is presenting — but it is an inference, not a directly observed result for that specific device, and is recorded as such rather than folded into the confirmed-live count.

Fixed the same day: rejections now log a reason server-side. None of the six rejections above printed anything to the server console beyond a bare `200 OK` — the JSON reason (`"Device missing registration..."`, `"Hardware MAC address signature mismatch"`, etc.) was returned to the client but never surfaced to whoever is running the server, meaning an instructor watching the console during a live session had no way to tell why a given student was rejected without checking that student's phone screen directly. `verify_checkin()` was edited to print the specific failing gate and its relevant detail (the mismatched MAC pair, the competing roll number in a multi-profile lockout, the exception type on a signature failure, or the measured RTT on a latency rejection) at each of its five return points. This is additive only — no return value, status code, or control flow changed — and was verified with a direct Python AST parse of the edited file to confirm no syntax errors were introduced. It has not yet been re-tested live against a new rejection.

9.5 Physical fencing and access-point selection

Finding 4.4 established that the latency and MAC-binding gates verify network membership, not physical presence — a device in the hallway or an adjacent room can pass both. Closing that gap was worked through as a design discussion rather than code, and it is recorded here because the reasoning ruled out two plausible ideas for concrete, evidence-based reasons rather than just intuition.

Ultrasonic proximity tokens were ruled out. The idea was a laptop-emitted, inaudible high-frequency tone picked up by the phone's microphone as proof of being in earshot. Acoustic attenuation in air increases sharply with frequency, so the higher a tone is pushed to stay inaudible, the faster it dies off with distance — dedicated ultrasonic transducers built for proximity sensing top out around 5–10 meters even in ideal conditions, and a laptop's built-in speaker is far weaker and less directional

than purpose-built hardware. For a 65-student room, let alone a larger one, back-row phones would not reliably receive it. This does not scale to classroom size and was dropped.

A rotating QR code helps with replay protection but not with fencing on its own. Displaying a QR on the projector that rotates every few seconds was considered as a physical-presence check. It is vulnerable to a **relay attack** (also called a wormhole attack in the security literature) — a well-known, unsolved-in-general problem for proximity systems generally, not specific to this project: a student inside the room points a phone camera at the projector and video-calls a friend outside, who scans directly off the live video feed. Rotation defeats a delayed screenshot but not a real-time relay, since video call codecs faithfully reproduce whatever is on screen. Because the attacker's device still has to reach 192.168.137.1 to submit anything, this specifically requires the outside accomplice to also be within Wi-Fi range — the same hallway/adjacent-room scope as Finding 4.4, not a truly remote attacker. The existing 20ms latency gate does not help here either: it measures the phone-to-laptop network round-trip, which looks identical whether the phone's owner is standing in the room or in the hallway relaying a video call, since the relay happens before any network request is made. A short rotation window (2–3 seconds) combined with a tight scan-to-submit time limit raises the cost of this attack from "screenshot and forward" to "live synchronized video relay," which is a real deterrent even though it is not a mathematical guarantee. The rotating QR remains worth building regardless, because it separately closes the replay gap in Section 5 — MOCK_CHALLENGE_FROM_QR was always meant to be a real per-session scan.

The most robust fencing mechanism found is physical: containing the Wi-Fi signal itself, using hardware the instructor owns. Rather than detecting presence after the fact, a personal access point with reduced transmit power and directional antenna placement can be tuned so its signal roughly fills the room rather than the hallway — standard practice in facilities that need to bound Wi-Fi coverage deliberately. This converges with Finding 4.5 and Finding 4.11: the same hardware change (replacing Windows Mobile Hotspot with a dedicated, instructor-owned access point) simultaneously fixes the 8-device cap, the port-53/ICS conflict from Section 9.1, the gateway-dependency requirement from Finding 4.11, and gives a path to physical containment. It is not precise — RF does not respect walls exactly, needs a per-room site survey rather than a single trusted setting, and does not stop someone carrying a second device physically into the room (a different attack, already caught by MAC binding and signature verification). The realistic posture is layered: RF containment as the primary boundary, the signed check-in pipeline underneath it, and a lightweight procedural spot-check by the instructor as a backstop for whatever slips through — not a single technical fix presented as airtight.

Access point candidates evaluated. A generic reseller-brand router ("PunnkFunnk WT-14", ~₹1,499) was considered first but could not be verified: the brand's other listed products are mostly Wi-Fi repeaters/extenders rather than full routers, one comparable listing in the same product line advertised only "10 devices, 10m coverage" — barely past the Windows Hotspot cap and short-range for a full classroom — and the steep listed discount (75% off an unverified MRP) is a common pattern with unverifiable generic electronics. Not recommended.

A TP-Link TL-WR845N (~₹1,300–1,500, 26,045 ratings, 4.1/5) was evaluated next. Correction to an earlier draft of this section: TP-Link's own documentation and support forums confirm this class of router **does** expose a Low/Middle/High transmit-power setting in stock firmware (roughly 10%/32%/100% of rated 2.4GHz output) — the physical-containment tuning discussed above is achievable without third-party firmware after all. What rules this specific model out is a different, better-documented limit: TP-Link's own community guidance recommends single-band routers in this class for roughly 20–25 concurrent clients, well short of a 65–200 student class, and it is 2.4GHz-only, so there is no 5GHz band to offload onto or to use as a natural range-limited channel.

Decision: a budget dual-band AC1200 router — Mercusys AC10 (~₹1,400, confirmed 3-year India warranty). Mercusys is TP-Link's budget sub-brand, built on the same underlying firmware family. The AC10 raises the client ceiling by splitting devices across two radios rather than one, and its 5GHz band offers a second, more direct containment lever than transmit power alone: 5GHz signals attenuate faster through walls than 2.4GHz, so a 5GHz-only SSID naturally stays closer to the room it's broadcast in. The practical plan is to run two SSIDs — a 5GHz network for modern phones (attenuates at the walls, higher client ceiling) and a 2.4GHz network left open for older phones that don't support 5GHz at all, low transmit power and physical placement applied to the 2.4GHz radio specifically since that is the one most likely to reach the hallway. One caveat carried over from the TL-WR845N evaluation and not yet re-verified on this specific model: confirm AP isolation

is off on the main network (not just assumed from the TP-Link/Mercusys family resemblance) with a real `arp -a` test per Finding 4.11 before relying on it for a class. Not yet purchased or tested.

10. How Far This Can Realistically Be Pushed

The underlying ideas — client-side asymmetric device binding, ARP-based link-layer correlation, and latency-based network gating — are legitimate and more sophisticated than a typical student attendance project, and this is no longer resting only on a single early solo run: a genuinely unconfigured, first-time device has now completed the full pipeline unassisted, which is meaningfully stronger evidence than what this report could say a day earlier. But “production-ready,” as the existing report's conclusion claims, is not an accurate description of the current state: the localStorage regression alone defeats the anti-spoofing thesis the whole architecture is built around, MAC randomization will silently break hardware binding for a large share of modern phones unless every student is walked through a manual settings change, and there is no persistence for certificates, no audit trail, and no automated test coverage. With the key-storage regression fixed, MAC randomization handled as an explicit onboarding step, a single authoritative codebase instead of three, and a real access point in place of Windows Mobile Hotspot, this is realistically a solid classroom-scale offline attendance tool — tens of students, one room, one instructor device. Scaling beyond a single access point's practical client ceiling, or across multiple rooms simultaneously, is not something the current architecture attempts, and would be a separate phase of work rather than a tuning problem.